Persistencia de Datos

- La **persistencia de datos** es la capacidad de un equipo para mantener la información incluso después de apagar el ordenador o cerrar un programa que utiliza la información.

- Una de las maneras más simples de mantener sus datos es a través de archivos de texto. Otra alternativa es usar Bases de Datos

Beneficios

- Permite que los datos que se generan una aplicación puedan ser almacenados en un archivo externo para poder ser utilizados posteriormente.

- Los datos persistentes pueden ser recuperados, modificados y almacenados nuevamente, conservando los cambios realizados de manera permanente.

Archivos de Texto

- Un archivo de texto almacena la información mediante la sucesión de caracteres que pueden considerarse como organizados en una secuencia de líneas de texto.
- Desde el punto de vista de programación un archivo de texto se considera como un objeto en el que se puede leer y escribir información.

Pasos al trabajar con archivos de texto

- Abrir el archivo
- Leer o escribir información en el archivo
- Cerrar el archivio

Manejo de archivos en Python

- La función clave para trabajar con archivos en Python es la función open().
- La función open() toma dos parámetros; nombre de archivo y modo

Hay cuatro métodos (modos) diferentes para abrir un archivo:

"r" - Leer: valor predeterminado. Abre un archivo para lectura, error si el archivo no existe

"a" - Agregar: abre un archivo para agregarlo, crea el archivo si no existe

"w" - Escribir: abre un archivo para escribir, crea el archivo si no existe

"x" - Crear: crea el archivo especificado, devuelve un error si el archivo existe

Además, puede especificar si el archivo debe manejarse en modo binario o de texto

"t" - Texto: valor predeterminado. Modo texto

"b" - Binario - Modo binario (por ejemplo, imágenes)

Lectura de un Archivo

Ejemplo de lectura de apertura de un archivo para lectura llamado ejemplo.txt. Se recomienda siempre hacerlo dentro de un try -catch, ya que si no existe genera un error.

```
try:
    archivo = open("ejemplo.txt","r")
    #acciones con el archivo
    lineas = archivo.readlines()
    for linea in lineas:
        print(linea)
    #cerrar el archivo
    archivo.close()
except IOError as error:
    print("Archivo no existe")
```

Ejemplo apertura de un archivo para lectura donde lee la primera línea y después imprimi cada carácter de la línea leída.

```
try:
    archivo = open("ejemplo.txt","r")
    #acciones con el archivo
    linea = archivo.readline()
    for caracter in linea:
        print(caracter)
    #cerrar el archivo
    archivo.close()
except IOError as error:
    print("Archivo no existe")
```

Leer un archivo y a cada línea se le elimina el carácter de salto de línera, uso del método ***strip***.

```
try:
    archivo = open(nombreArchivo,"r")
    #acciones con el archivo
    lineas = archivo.readlines()
    for linea in lineas:
        p = linea.strip("\n")
        print(p)
    #cerrar el archivo
    archivo.close()
except IOError as error:
    print("Archivo no existe")
```

Leer un archivo para lectura donde cada línea se divide de acuerdo a un carácter que se encuentre en la línea. En el ejemplo el carácter es el **;**

```
try:
    archivo = open(nombreArchivo,"r")
    #acciones con el archivo
    lineas = archivo.readlines()
    for linea in lineas:
        p = linea.strip("\n").split(";")
        print(p)
    #cerrar el archivo
    archivo.close()
except IOError as error:
    print("Archivo no existe")
```

Escribir en un Archivo de Texto

Para escribir en un archivo existente, debe agregar un parámetro a la función open():

"a" - Agregar: se agregará al final del archivo

"w" - Escribir: sobrescribirá cualquier contenido existente

Ejemplo apertura de un archivo de texto para agregarle información

De este modo si el archivo no existe se crea, y si ya existe se agrega la información al final del archivo.

```
archivo = open("ciudades.txt","a")
archivo.write("Neiva\n")
archivo.write("Popayán\n")
archivo.close()
```

Ejemplo apertura de un archivo de texto para escribir modo sobreescribir

De este modo si el archivo no existe se crea y si existe se elimina su contenido y se sobreescribe con la nueva información.

```
archivo = open("ciudades.txt","w")
archivo.write("Neiva\n")
archivo.write("Popayán\n")
archivo.close()
```

Crear un Archivo de Texto

Para crear un nuevo archivo en Python, use el método open(), con uno de los siguientes parámetros:

"x" - Crear: creará un archivo, devuelve un error si el archivo existe

"a" - Adjuntar: creará un archivo si el archivo especificado no existe

"w" - Escribir: creará un archivo si el archivo especificado no existe

Crear un archivo método X: Genera una excepción si el archivo existe

```
try:
    archivo = open("ejemplo.txt","x")
    #acciones con el archivo
    print("Archivo nuevo creado")
    archivo.close()
except IOError as error:
    print("El archivo ya existe")
```

Eliminar un archivo

Para eliminar un archivo, debe importar el módulo del sistema operativo y ejecutar la función os.remove(). Se recomienda primero verificar si el archivo existe como se muestra a continuación.

```
import os

if os.path.exists("ejemplo.txt"):
  os.remove("ejemplo.txt")
else:
  print("El archivo no existe")
```

Referencias:

- Tutorial Python: https://www.w3schools.com/python
- Documentación oficial Python: https://docs.python.org/es/3.9/library/io.html?highlight=working%20streams

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CIES-NEIVA	2-7-2021